

SRS PawMily

SRS - Especificación de Requerimientos de Software

Sistema PawMily

****Versión:**** 1.0
****Fecha:**** 25 de agosto de 2026
****Alcance:**** Backend API, portal web (veterinario), aplicación Android (propietario)

1. Introducción

1.1 Propósito

Definir los requerimientos funcionales (RF) y no funcionales (RNF) del sistema PawMily para desarrollo, verificación y trazabilidad.

1.2 Alcance del producto

PawMily permite a veterinarios gestionar pacientes, consultas, dieta, agenda y configuración de clínica; y a propietarios vincular mascotas por código/barcode, consultar expediente, alimentación, historial y gestionar recordatorios personales.

1.3 Definiciones

Término | Definición
Vet | Usuario rol `vet` (clínica)
Owner | Usuario rol `owner` (dueño)
Patient / Paciente | Mascota con código `PAW-?`
Link | Solicitud/vinculación owner?paciente
JWT | Token de autenticación de la API

1.4 Referencias

- Código backend Express Clean/DDD
- Prisma `schema.prisma`
- App Android `RemotePetRepository` / Retrofit
- Web `js/api.js` (PawApi)

2. Descripción general

2.1 Perspectiva del producto

Sistema distribuido en tres clientes + API + BD:

Capa | Tecnología | Estado
API | Express + TS + Prisma + Zod + JWT | ****Producción (Railway)****
BD | PostgreSQL (Supabase) | ****Activa****
Web | HTML/JS estático | ****Operativa (Vercel)****
Android | Kotlin + Retrofit | ****Operativa vs API prod****

2.2 Características principales

1. Autenticación y perfil

2. CRUD pacientes (vet) y vínculo por código (owner)
3. Historial médico / consultas
4. Alimentación / dieta
5. Recordatorios
6. Citas / agenda
7. Configuración de clínica
8. Código de barras Code128 / lector HID
9. Favoritos (app) y media (URLs firmadas)

2.3 Usuarios

Actor | Canal | Capacidades

Veterinario | Web | Pacientes, consultas, dieta, agenda, config, aprobar vínculos OWNER

Propietario | Android | Registro/login, vincular, ver resumen/historial/alimentación, recordatorios, confirmar citas

Sistema | API | Auth, autorización por rol, auditoría parcial

3. Requerimientos funcionales

3.1 Autenticación e identidad

ID | Requerimiento | Prioridad | Estado

RF-AUTH-01 | Registro de usuario con email/contraseña y rol | Alta | Implementado

RF-AUTH-02 | Login emitiendo access + refresh token | Alta | Implementado

RF-AUTH-03 | Refresh de access token | Alta | Implementado (API + Android)

RF-AUTH-04 | Logout invalidando refresh | Media | Implementado

RF-AUTH-05 | Consulta y actualización de perfil | Alta | Implementado

RF-AUTH-06 | Solicitud/confirmación de verificación de email | Media | Implementado API

RF-AUTH-07 | Recuperación de contraseña | Media | Implementado API / web parcial

3.2 Pacientes y vinculación

ID | Requerimiento | Prioridad | Estado

RF-PAT-01 | Vet crea paciente con código seguro `PAW-?` | Alta | Implementado

RF-PAT-02 | Vet lista/busca/edita/elimina pacientes propios | Alta | Implementado

RF-PAT-03 | Generación/consulta de barcode Code128 | Alta | Implementado

RF-PAT-04 | Owner solicita vínculo por código | Alta | Implementado

RF-PAT-05 | Aprobación/rechazo de solicitudes de vínculo | Alta | Implementado

RF-PAT-06 | Owner lista mascotas vinculadas (`/patients/mine`) | Alta | Implementado

RF-PAT-07 | Owner desvincula paciente | Media | Implementado

RF-PAT-08 | Consulta de miembros/accesos del paciente | Media | Implementado API

3.3 Historial médico

ID | Requerimiento | Prioridad | Estado

RF-MED-01 | Vet crea/edita/elimina registros médicos | Alta | Implementado

RF-MED-02 | Vet y owner (autorizado) consultan historial | Alta | Implementado

3.4 Alimentación

ID | Requerimiento | Prioridad | Estado

RF-FED-01 | Vet define dieta del paciente | Alta | Implementado

RF-FED-02 | Owner consulta alimentación | Alta | Implementado

RF-FED-03 | Owner actualiza campos descriptivos de feeding | Media | Implementado

RF-FED-04 | Registro de logs de comida | Media | Implementado API

3.5 Recordatorios

ID | Requerimiento | Prioridad | Estado

RF-REM-01 | CRUD recordatorios asociados a paciente | Alta | Implementado

RF-REM-02 | Completar recordatorio | Media | Implementado

RF-REM-03 | Prioridad / favoritos (estrellas) en app | Media | Implementado Android

RF-REM-04 | Notificaciones locales en Android | Media | Implementado (AlarmManager)

3.6 Citas

ID | Requerimiento | Prioridad | Estado

RF-APT-01 | Vet CRUD citas y vista mensual | Alta | Implementado

RF-APT-02 | Owner lista citas propias | Alta | Implementado

RF-APT-03 | Owner confirma asistencia | Media | Implementado

3.7 Clínica y media

ID | Requerimiento | Prioridad | Estado

RF-CFG-01 | Vet consulta/actualiza ClinicConfig | Media | Implementado

RF-MEDA-01 | Obtener URL de media / upload firmado | Media | Implementado API

3.8 Clientes (canales)

ID | Requerimiento | Prioridad | Estado

RF-WEB-01 | Portal vet: login, dashboard, pacientes, agenda, config | Alta | Implementado

RF-WEB-02 | Lectura HID de códigos en web | Media | Implementado

RF-AND-01 | App owner: home, mascotas, perfil, recordatorios | Alta | Implementado

RF-AND-02 | Escaneo cámara + vínculo por código | Alta | Implementado

RF-AND-03 | Perfil de mascota: resumen, historial, alimentación | Alta | Implementado

4. Requerimientos no funcionales

ID | Categoría | Requerimiento | Estado

RNF-01 | Seguridad | HTTPS en producción; JWT Bearer; rate limit en `/api` y auth | Cumple

RNF-02 | Seguridad | Contraseñas con bcrypt; refresh tokens hasheados | Cumple

RNF-03 | Disponibilidad | API desplegada en Railway con health check | Cumple

RNF-04 | Escalabilidad | Stateless API; BD gestionada Supabase | Cumple (básico)

RNF-05 | Mantenibilidad | Clean Architecture + DDD en backend | Cumple

RNF-06 | Usabilidad | UI web clínica; app Android material | Cumple parcial

RNF-07 | Interoperabilidad | Contrato REST JSON único para web y Android | Cumple

RNF-08 | Observabilidad | Request logger; audit logs en BD | Cumple parcial

RNF-09 | Rendimiento | Listados lean de pacientes; caché en Android | Cumple parcial

RNF-10 | Privacidad | Aislamiento por vetId / ownerId / PatientAccess | Cumple

5. Restricciones y supuestos

- No hay rol `admin` de plataforma; administración = vet + soporte técnico.
- Identificador visible de mascota: código `PAW-?` (no QR de expediente).
- Fotos pueden almacenarse como data-URL o `photoAssetId` + storage.
- Backend local puede no estar versionado en git remoto.

6. Criterios de aceptación globales

1. Health de API responde `ok`.
2. Vet puede crear paciente y verlo en web.
3. Owner puede vincular por código y ver perfil en Android.
4. Historial y alimentación se leen desde API (sin mock).
5. Tokens se refrescan sin forzar login inmediato en cada 401 recuperable.